

Labor ZH - Teljes, „hülyebiztos” megoldási segédlet

Mi ez? Lépésről lépésre haladó dokumentáció a labor ZH összes feladattípusához. Úgy van felépítve, hogy a vizsga közben fentről lefelé haladva végig tudd csinálni. Minden nagyobb lépés után van egy  **Ellenőrzés** pont – ha az nem sikerül, NE menj tovább, hanem nézd meg a  **Hibaelhárítás** részt.

Fontos: Nem ismerem a te konkrét VM-edet (elérési utak, jelszavak, verziók eltérhetnek). Ahol ez számít, ott megmutatom, **hogyan derítsd ki a valós értéket**. A példában szereplő útvonalakat (`/etc/filebeat/...` stb.) mindig vedd össze a saját géped valóságával.

0. A vizsga áttekintése és az ajánlott sorrend

A feladatok **egymásra épülnek**. Ezért ezt a sorrendet javaslom:

1. **Adatbeküldés** (Filebeat → Logstash → OpenSearch) — ez minden más alapja.
2. **Grok parsing** (Logstash filter) — a strukturált mezők kellenek a dashboardhoz, riasztáshoz, elemzéshez.
3. **Index pattern + Dashboardok** — ehhez már kell beérkezett, strukturált adat.
4. **Riasztás (Teams webhook)** — ehhez is kell adat és mező.
5. **Logelemzés** — a beérkezett adatból Discoverrel/DQL-lel.
6. **Forgalomelemzés (PCAP)** — ez teljesen független, Wireshark/tshark, bármikor csinálható.

 **Tipp:** A PCAP-elemzés (6.) független mindentől. Ha valahol elakadsz az ELK/OpenSearch résznél, ugorj a PCAP-re, szerezd meg azokat a pontokat, aztán térj vissza.

1. Előkészületek - szolgáltatások és hozzáférések ellenőrzése

Mielőtt bármit konfigurálnál, győződj meg róla, hogy minden szolgáltatás fut és tudod, hol vannak a fájlok.

1.1 Szolgáltatások státusza

```
bash

# Fő szolgáltatások állapota
sudo systemctl status opensearch
sudo systemctl status opensearch-dashboards
sudo systemctl status logstash
sudo systemctl status filebeat
```

Ha valamelyik nem fut, indítsd el:

```
bash

sudo systemctl start opensearch
sudo systemctl start opensearch-dashboards
sudo systemctl start logstash
# A filebeat-et szándékosan csak akkor indítjuk, amikor a configot már beállítottuk
```

⚠ Az OpenSearch indulása **lassú** lehet (akár 1-2 perc). Légy türelmes.

1.2 Hol vannak a fontos fájlok? (derítsd ki, ne tippelj!)

```
bash

# Konfigurációs fájlok megkeresése
ls -la /etc/filebeat/filebeat.yml
ls -la /etc/logstash/conf.d/           # itt vannak a Logstash pipeline-ok (*.conf)
ls -la /etc/logstash/pipelines.yml
ls -la /etc/opensearch/opensearch.yml

# Ha nem ott vannak, keresd meg:
sudo find / -name "filebeat.yml" 2>/dev/null
sudo find / -name "*.conf" -path "*logstash*" 2>/dev/null
```

A **log-fájlok** (amiket be kell küldeni) helyét a feladatléírás adja meg. Ha nem, keresd:

```
bash

ls -la /var/log/
sudo find / -name "*.log" -newermt "2020-01-01" 2>/dev/null | head -50
```

1.3 OpenSearch elérés és jelszó

Az OpenSearch a biztonsági plugin miatt **HTTPS-en**, a **9200**-as porton érhető el, felhasználóval/jelszóval.

```
bash

# Próbáld az alapértelmezett admin/admin-t (régebbi verzió):
curl -k -u admin:admin https://localhost:9200

# Ha 401 (Unauthorized) jön, a 2.12+ verzió egyedi jelszót kér.
# Nézd meg, beállították-e környezeti változóként vagy a doksiban:
sudo grep -ri "INITIAL_ADMIN_PASSWORD" /etc/ 2>/dev/null
cat /etc/opensearch/opensearch.yml | grep -i pass
```

A **-k** kapcsoló az önaláírt tanúsítvány miatt kell (kikapcsolja a cert-ellenőrzést).

1.4 OpenSearch Dashboards (a webes felület)

Böngészőben: (`http://localhost:5601`) (vagy a VM IP-je (`:5601`)). Bejelentkezés ugyanazzal a felhasználó/jelszó párral (pl. `admin`) / a beállított jelszó).

✅ **Ellenőrzés:** Be tudsz lépni a Dashboards felületre, és a (`curl ... https://localhost:9200`) választ ad (nem connection refused). Ha igen, mehetsz tovább.

2. FELADAT: Logok beküldése Filebeat → Logstash → OpenSearch

Cél: A Filebeat olvassa a megadott log-fájl(oka)t, elküldi a Logstash-nek (5044-es port), a Logstash pedig betölti az OpenSearch-be.

```
[log fájl] --> Filebeat --(5044/beats)--> Logstash --(9200/https)--> OpenSearch
```

2.1 Logstash pipeline - először a Logstash legyen kész (ez „fogadja” a Filebeatet)

Nyisd meg / hozd létre a pipeline configot (a fájlnev lehet más, pl. `pipeline.conf`, `beats.conf`):

```
bash  
  
sudo nano /etc/logstash/conf.d/pipeline.conf
```

Egy **minimális, működő** pipeline (parsing nélkül – azt a 3. fejezetben tesszük bele):

```
ruby
```

```

input {
  beats {
    port => 5044
  }
}

filter {
  # Egyelőre üres - a grokot a 3. fejezetben tesszük ide.
}

output {
  opensearch {
    hosts          => ["https://localhost:9200"]
    index          => "zh-logs-%{+YYYY.MM.dd}"
    user           => "admin"
    password       => "admin"          # <-- ha más a jelszó, írd át!
    ssl            => true
    ssl_certificate_verification => false
  }

  # Hibakereséshez NAGYON hasznos: kiírja a konzolra, mit dolgoz fel.
  stdout {
    codec => rubydebug
  }
}

```

🔑 **Kulcsfontosságú a `stdout { codec => rubydebug }`!** Ettől látod a Logstash konzolján, hogy pontosan milyen mezőket állít elő. A grok hibakeresés itt fog történni. A vizsga végén kivetheted, de fejlesztés közben hagyd benn.

Ellenőrizd, hogy az OpenSearch output plugin telepítve van-e

```

bash

sudo /usr/share/logstash/bin/logstash-plugin list | grep opensearch

```

Ha nem jelenik meg a `logstash-output-opensearch`, telepítsd:

```

bash

sudo /usr/share/logstash/bin/logstash-plugin install logstash-output-opensearch

```

(Az OpenSearch által terjesztett „Logstash OSS with OpenSearch output plugin” csomagban ez már eleve benne van.)

Teszteld a config szintaxisát, mielőtt elindítod

```
bash
```

```
sudo -u logstash /usr/share/logstash/bin/logstash \  
  --path.settings /etc/logstash \  
  -f /etc/logstash/conf.d/pipeline.conf \  
  --config.test_and_exit
```

Ha a végén `Configuration OK`, jó. Ha hibát ír, az megmondja, melyik sorban van a gond (általában hiányzó `}` vagy `=>`).

Indítsd újra a Logstash-t

```
bash
```

```
sudo systemctl restart logstash
```

```
# Nézd a logját előben (ide jönnek a hibák és a rubydebug kimenet, ha foreground-ban  
sudo tail -f /var/log/logstash/logstash-plain.log
```

💡 **Alternatíva fejlesztéshez:** futtasd a Logstash-t **előtérben (foreground)**, így egyből a terminálban látod a `rubydebug` kimenetet:

```
bash
```

```
sudo systemctl stop logstash  
sudo -u logstash /usr/share/logstash/bin/logstash \  
  --path.settings /etc/logstash \  
  -f /etc/logstash/conf.d/pipeline.conf
```

A leállítás `Ctrl+C`. A vizsga végére kapcsolj vissza systemd-vel, ha az kell.

✅ **Ellenőrzés:** A logban megjelenik valami ilyesmi: `Beats inputs: Starting input listener { :address=>"0.0.0.0:5044" }` és nincs `ERROR`. Ellenőrizd, hogy a port nyitva van:

```
bash
```

```
sudo ss -tlnp | grep 5044
```

2.2 Filebeat konfigurálása

Nyisd meg a configot:

```
bash
```

```
sudo nano /etc/filebeat/filebeat.yml
```

A lényeg három dolog: **(a)** honnan olvasson, **(b)** a Logstash output legyen bekapcsolva, **(c)** az OpenSearch/Elasticsearch közvetlen output legyen Kikapcsolva.

```
yaml

# ===== (a) BEMENET: melyik fájlokat olvassa =====
filebeat.inputs:
  - type: filestream          # újabb (8.x) verzióban "filestream"; régebbiben (7.x) "
    id: zh-input
    enabled: true
    paths:
      - /var/log/minta/*.log    # <-- ÍRD ÁT a feladatban megadott útvonalra!
      # - /eleresi/ut/a/logokhoz/*.log

# ===== (b) KIMENET: Logstash =====
output.logstash:
  hosts: ["localhost:5044"]

# ===== (c) Ezt KAPCSOLD KI (tedd kommentbe a teljes blokkot)! =====
# output.elasticsearch:
#   hosts: ["localhost:9200"]
```

⚠ **Gyakori buktató:** A `filebeat.yml`-ben **csak EGY** output lehet aktív. Ha az `output.elasticsearch` is be van kapcsolva (nincs kikommentelve), a Filebeat hibát dob, vagy nem a Logstash-be küld. Győződj meg róla, hogy az `output.elasticsearch` minden sora `#`-tel kezdődik.

⚠ **YAML behúzás:** A YAML-ben a szóközös behúzás kötelező, TAB **tilos**. Ha „could not parse config” hibát kapsz, szinte biztosan behúzási hiba van.

Teszteld a Filebeat configot

```
bash

sudo filebeat test config -c /etc/filebeat/filebeat.yml
# Várt válasz: Config OK

sudo filebeat test output -c /etc/filebeat/filebeat.yml
# Várt válasz: logstash: localhost:5044... talk to server... OK
```

Ha az output teszt nem OK → a Logstash nem fut, vagy az 5044 nincs nyitva → vissza a 2.1-hez.

Indítsd el a Filebeat-et

A legjobb fejlesztéshez **előtérben**, hogy lásd a logokat:

```
bash
```

```
sudo filebeat -e -c /etc/filebeat/filebeat.yml
```

(-e) = logok a konzolra). Leállítás: (Ctrl+C). Ha systemd-vel akarod:

```
bash
```

```
sudo systemctl restart filebeat
```

```
sudo journalctl -u filebeat -f      # logok élőben
```

✅ Ellenőrzés (a teljes lánc!):

1. A **Logstash** konzolján/logjában megjelennek a (rubydebug) események (egy-egy log sor szétbontva mezőkre, benne (message), (@timestamp), (host) stb.).
2. Az OpenSearch-ben létrejött az index, és van benne dokumentum:

```
bash
```

```
curl -k -u admin:admin "https://localhost:9200/_cat/indices?v"
```

```
# Keresd a "zh-logs-..." indexet, a docs.count > 0 legyen.
```

```
# Egy minta dokumentum megtekintése:
```

```
curl -k -u admin:admin "https://localhost:9200/zh-logs-*/_search?pretty&size=1"
```

Ha mindkettő igaz: az adatbeküldés kész. 🎉

2.3 ⚠️ A LEGGYAKORIBB BUKTATÓ: a Filebeat „registry” (újraküldés)

A Filebeat **megjegyzi**, meddig olvasott el egy fájlt (registry). Ha egyszer már beolvasta a logot, **másodszorra NEM küldi újra** – akkor sem, ha közben átírtad a Logstash grokot, és újra szeretnéd látni az adatokat.

Ha újra be akarod küldeni ugyanazokat a logokat (pl. mert javítottad a parsingot, és tiszta indexet akarsz):

```
bash
```

```
# 1) Állítsd le a Filebeat-et
sudo systemctl stop filebeat
# (vagy Ctrl+C, ha előtérben fut)

# 2) Töröld a registry-t (a registry pontos helye lehet más, ezért keressük is)
sudo rm -rf /var/lib/filebeat/registry
# Ha máshol van: sudo find / -type d -name registry -path "*filebeat*" 2>/dev/null

# 3) (opcionális) Töröld a régi indexet az OpenSearch-ben tiszta lappal:
curl -k -u admin:admin -X DELETE "https://localhost:9200/zh-logs-*"

# 4) Indítsd újra a Filebeat-et
sudo filebeat -e -c /etc/filebeat/filebeat.yml
```

🧠 Jegyezd meg ezt a 4 lépést. Vizsgán a parsing finomhangolása közben sokszor kell majd „nulláról” újraküldeni.

3. FELADAT: Strukturálás (parsing) grok filterrel a Logstash-ben

Cél: A nyers `message` mezőt szétbontani külön mezőkre (pl. IP, HTTP-státusz, URL, időbélyeg), hogy szűrhető/aggregálható legyen.

3.1 A munkamódszer (ez a kulcs!)

1. Nézz meg **egy nyers log sort**, hogy lásd a formátumát:

```
bash

head -n 5 /var/log/minta/minta.log
```

2. Írj rá egy grok mintát a Logstash `filter`-ben.
3. Indítsd újra a Logstash-t, **küldd újra a logot** (lásd 2.3), és nézd a `rubydebug` kimenetet.
4. Ha a `tags` mezőben ott van `_grokparsefailure` → **a minta nem illeszkedik**, finomíts rajta.
5. Ismételd, amíg minden mező szépen kiválik és nincs parse failure.

3.2 A grok filter beillesztése

A `filter { }` blokkba (a `pipeline.conf`-ban):

```
ruby
```



```

filter {
  grok {
    match => { "message" => "MINTA_IDE" }
  }

  # Az időbélyeget állítsuk be a @timestamp-ra (lásd 3.5):
  # date { ... }

  # Típuskonverzió, mezőtörlés, átnevezés (lásd 3.6):
  # mutate { ... }
}

```

3.3 Kész grok minták gyakori log-típusokhoz

Apache / Nginx access log (a leggyakoribb a ZH-kban):

```

ruby

filter {
  grok {
    match => { "message" => "%{COMBINEDAPACHELOG}" }
  }
}

```

Ez kiadja többek közt: `clientip`, `ident`, `auth`, `timestamp`, `verb` (GET/POST), `request` (URL), `httpversion`, `response` (státuszkód), `bytes`, `referrer`, `agent` (user agent).

Megjegyzés: egyes verziókban a minta neve `%{HTTPD_COMBINEDLOG}` vagy `%{HTTPD_COMMONLOG}`. Ha a `COMBINEDAPACHELOG` ismeretlen mintát jelez, próbáld ezeket.

Syslog sor:

```

ruby

filter {
  grok {
    match => { "message" => "%{SYSLOGTIMESTAMP:syslog_timestamp} %{SYSLOGHOST:syslog_host} %{SYSLOGMESSAGE:syslog_message}" }
  }
}

```

SSH bejelentkezés (auth.log) – sikertelen jelszó:

```

ruby

```

```
filter {
  grok {
    match => { "message" => "Failed password for (invalid user )?%{USERNAME:username}"
  }
}
```

Saját formátum „kézzel” összerakva (a leggyakrabban használt elemi minták):

Minta	Mit fog	Példa
<code>%{IP:ip}</code>	IPv4/IPv6 cím	<code>192.168.1.10</code>
<code>%{IPORHOST:host}</code>	IP vagy hostnév	<code>web01</code>
<code>%{NUMBER:num}</code>	szám	<code>404</code> , <code>1.5</code>
<code>%{INT:n}</code>	egész szám	<code>42</code>
<code>%{WORD:w}</code>	egy szó (betű/szám)	<code>GET</code>
<code>%{NOTSPACE:x}</code>	bármilyen szóköz	<code>/index.html</code>
<code>%{DATA:x}</code>	bármilyen (mohótlán, rövid)	használd két fix elhatároló közé
<code>%{GREEDYDATA:x}</code>	minden a sor végéig	a maradék szöveg
<code>%{QS:x}</code>	idézőjelbe tett szöveg	<code>"Mozilla/5.0 ..."</code>
<code>%{TIMESTAMP_ISO8601:ts}</code>	ISO idő	<code>2024-03-01T12:00:00Z</code>
<code>%{HTTPDATE:ts}</code>	Apache idő	<code>01/Mar/2024:12:00:00 +0100</code>
<code>%{LOGLEVEL:level}</code>	szint	<code>ERROR</code> , <code>INFO</code>

Példa egy egyedi sorra (`2024-03-01 12:00:00 ERROR 10.0.0.5 /login 500`):

```
ruby

grok {
  match => { "message" => "%{TIMESTAMP_ISO8601:ts} %{LOGLEVEL:level} %{IP:client_ip}"
}
```

3.4 Grok mintatesztelő eszközök

A leggyorsabb, **mindig működő** módszer: a Logstash `stdout { codec => rubydebug }` kimenete (lásd 2.1). Indítod, újraküldöd a logot, nézed a mezőket.

Online grok-tesztelő is használható (ha a labor enged netet): keresd a „grok debugger” oldalt, illeszd be a log sort és a mintát, addig faragod, míg minden mező kijön. Másold be a kész mintát a configba.

3.5 Időbélyeg helyes beállítása (`date` filter) - FONTOS!

Alapból a `@timestamp` az lesz, amikor a Logstash feldolgozta a sort - **nem** a logban szereplő idő. Ez tönkreteszi az időalapú dashboardot/elemzést. Ezért a logból kinyert időt rá kell tenni a `@timestamp`-ra:

```
ruby

filter {
  grok { match => { "message" => "%{COMBINEDAPACHELOG}" } }

  date {
    match => [ "timestamp", "dd/MMM/yyyy:HH:mm:ss Z" ] # Apache formátum
    target => "@timestamp"
  }
}
```

ISO időhöz: `match => [ "ts", "ISO8601" ]`.

💡 Ha a Discoverben „No results found”, de tudod, hogy van adat → szinte biztos, hogy a logok régi dátumúak, és a `@timestamp` a beolvasás ideje. Vagy állítsd be a `date` filtert, vagy bővítsd ki a Dashboards időablakát (lásd 5.2).

3.6 Mezők rendezése: `mutate` (típus, átnevezés, törlés)

```
ruby

filter {
  grok { match => { "message" => "%{COMBINEDAPACHELOG}" } }
  date { match => [ "timestamp", "dd/MMM/yyyy:HH:mm:ss Z" ] }

  mutate {
    convert => { "response" => "integer" } # státusz legyen szám (így tudsz >= 500
    convert => { "bytes" => "integer" }
    rename => { "clientip" => "src_ip" } # átnevezés szebb névre
    remove_field => [ "ident", "auth" ] # felesleges mezők törlése
  }
}
```

💡 A `convert` azért fontos, mert a grok **mindent stringként** ad ki. Ha a státuszkódra tartomány-szűrést akarsz (`response >= 400`), előbb `integer`-ré kell alakítani.

(Opcionális, ha van geoip plugin és nyilvános IP-k vannak):

```
ruby

geoip { source => "src_ip" }           # ország, város, koordináták hozzáadása
```

3.7 Parsing utáni teljes újraküldés

A grok módosítása után:

```
bash

sudo systemctl restart logstash      # vagy Ctrl+C + újraindít előtérben
# majd a 2.3 szerinti registry-törlés + index-törlés + filebeat újraindítás
```

✅ **Ellenőrzés:** A `rubydebug` kimenetben (vagy egy `_search`-ben) ott vannak a **külön mezők** (pl. `src_ip`, `response`, `verb`), a `@timestamp` a logbeli idő, és **nincs** `_grokparsefailure` a `tags`-ben:

```
bash

# Hány parse hiba van? (0 a cél)
curl -k -u admin:admin "https://localhost:9200/zh-logs-*/_count?q=tags:_grokparsefai
```

4. FELADAT: Index pattern + Dashboardok az OpenSearch-ben

A dashboardhoz **először index pattern** kell, mert anélkül a Dashboards nem „látja” az adatot.

4.1 Index pattern létrehozása

1. OpenSearch Dashboards (`http://localhost:5601`) → bal felső **☰ menü** → **Dashboards Management** (régebbi verzióban: **Stack Management**) → **Index Patterns** (vagy **Index patterns**).
2. **Create index pattern.**
3. Írd be: `zh-logs-*` (a csillaggal együtt!). Ekkor lent meg kell jelennie az illeszkedő indexnek.
4. **Next step** → **Time field:** válaszd a `@timestamp`-ot.
5. **Create index pattern.**

✅ **Ellenőrzés:** A mezők listája megjelenik (pl. `src_ip`, `response`, `verb`, `@timestamp`).

4.2 Vizualizációk készítése

A dashboard vizualizációkból áll. Készíts párat:

1. **☰ menü** → **Visualize** → **Create visualization** → válaszd a típust.
2. Válaszd a `zh-logs-*` index patternt.

Példa A - Idősoros oszlopdiagram (forgalom időben):

- Típus: **Vertical Bar**
- **Metrics** → Y-axis: `Count`
- **Buckets** → X-axis: **Date Histogram**, mező: `@timestamp`
- **Update** → **Save** (adj nevet, pl. „Forgalom időben”).

Példa B - Top 10 forrás IP (táblázat):

- Típus: **Data Table**
- **Metrics**: `Count`
- **Buckets** → **Split rows: Terms**, mező: `src_ip` (vagy `clientip`), **Size**: 10, rendezés Count szerint csökkenő.
- **Save** → „Top forrás IP-k”.

Példa C - HTTP státusz kódok megoszlása (kör/pie):

- Típus: **Pie**
- **Metrics**: `Count`
- **Buckets** → **Split slices: Terms**, mező: `response`, **Size**: 10.
- **Save** → „Státusz kódok”.

Példa D - Egyetlen szám (Metric):

- Típus: **Metric**
- **Metrics**: `Count` → ez kiírja az összes esemény számát.
- **Save** → „Összes esemény”.

💡 Ha a feladat konkrét vizualizációt kér (pl. „a hibás kérések száma 5 percenként”), gondold végig: **mit mérünk** (Y/metric: Count vagy Average...) és **mi szerint bontjuk** (X/bucket: Date Histogram vagy Terms egy mezőn). Szükség esetén a vizualizáció keresőjébe írd DQL szűrőt (pl. `response >= 400`).

4.3 Dashboard összerakása

1. ☰ menü → **Dashboard** → **Create new dashboard** (vagy **Create dashboard**).
2. **Add** (jobb felső) → a listából kattints rá a korábban mentett vizualizációkra → bekerülnek a dashboardra.
3. Húzd/méretezd őket elrendezésbe.
4. Állítsd be jobb felül az **időablakot** úgy, hogy lásd az adatot (pl. „Last 1 year” vagy abszolút tartomány - lásd 5.2).
5. **Save** (adj nevet, pl. „ZH Dashboard”).

✅ **Ellenőrzés:** A dashboardon megjelennek az adatok (nem üresek a panelek). Ha üresek → időablak (5.2) vagy nincs adat (2-3. fejezet).

5. FELADAT: Logelemzés (kérdések megválaszolása az adatból)

Eszköz: a **Discover** és a **DQL** (Dashboards Query Language), illetve a fenti Data Table vizualizációk a „top N” típusú kérdésekhez.

5.1 Discover megnyitása

☰ menü → **Discover** → fent válaszd a `zh-logs-*` index patternt.

5.2 ⚠️ A LEGGYAKORIBB BUKTATÓ: az időablak

Ha „**No results found**”, de van adat: a Discover alapból csak az utolsó 15 percet mutatja. A mintalogok dátuma viszont gyakran régi. **Bővítsd ki az időablakot** (jobb felső sarok):

- Kattints az időválasztóra → **Last 1 year** (vagy 5 év), vagy
- **Absolute** → adj meg egy tág tartományt (pl. 2000 – ma).

5.3 DQL – szűrés a keresősávban

Kérdés típusa	DQL példa
Adott státuszkód	<code>response: 404</code>
Tartomány	<code>response >= 500</code>
Adott IP	<code>src_ip: "10.0.0.5"</code>
Adott HTTP metódus	<code>verb: "POST"</code>
URL minta	<code>request: "/admin*"</code>
Szövegrész egy mezőben	<code>agent: *curl*</code>
ÉS / VAGY / NEM	<code>response >= 400 and verb: "POST", status: 404 or status: 500, not src_ip: "10.0.0.1"</code>
Mező létezik-e	<code>src_ip: *</code>

💡 **Mezőnevek kiderítése:** Nyiss meg egy dokumentumot a Discoverben (kattints egy sorra → kinyílik), és látod az összes mezőnevet az értékkel. A bal oldali mezőlistából kattintással oszlopként is hozzáadhatod őket a táblázathoz.

5.4 „Top N” / „hány darab” típusú kérdések

Ezekre a leggyorsabb a **Data Table** vagy **Pie** vizualizáció **Terms** aggregációval (lásd 4.2/B-C), vagy a Discoverben a bal oldali mező melletti **Visualize**.

Tipikus ZH-kérdések és megoldásuk:

Kérdés	Megoldás
„Hány esemény érkezett összesen?”	Discover találati szám felül, vagy Metric vizualizáció (Count).
„Melyik IP küldte a legtöbb kérést?”	Data Table: Terms <code>src_ip</code> , rendezés Count szerint, az első sor.
„Hány 404-es hiba volt?”	DQL: <code>response: 404</code> → találati szám.
„Melyik a leggyakoribb URL?”	Data Table: Terms <code>request</code> .
„Volt-e bejelentkezési próbálkozás X felhasználóra?”	DQL: <code>username: "x"</code> .
„Mikor (mely órában) volt a csúcsforgalom?”	Date Histogram vizualizáció, a legmagasabb oszlop.

 **Írd le a választ azonnal** a kérdés mellé (érték + ahogy megkaptad), hogy a végén ne kelljen újra kikeresni.

6. FELADAT: Riasztás Teams webhook segítségével (OpenSearch Alerting)

Cél: Az OpenSearch figyeljen egy feltételt (pl. túl sok hibás kérés), és ha teljesül, küldjön üzenetet egy Teams csatornába webhookon át.

```
OpenSearch Monitor --> Trigger (feltétel) --> Action --> Notification Channel  
(webhook) --> Teams
```

6.1 ⚠ Fontos a Teams webhookról (2025–2026 változás)

A Microsoft **kivezette a régi „Incoming Webhook” connectort** (Office 365 / M365 Connector). Helyette a **Workflows (Power Automate)** webhook a hivatalos megoldás, ami **Adaptive Card** formátumú üzenetet vár (a régi connector „MessageCard”-ot várt). Ez azért fontos, mert befolyásolja, melyik OpenSearch csatornatípust kell választani.

Két foratókönyv:

- (A) A labor egy **kész webhook URL-t** ad neked, vagy **klasszikus connector** webhook → használd az OpenSearch natív „**Microsoft Teams**” csatornatípusát (legegyszerűbb).
- (B) Neked kell **Workflows** webhookot csinálni Teamsben → a natív „Microsoft Teams” csatorna üzenete lehet, hogy nem jelenik meg (formátumkülönbség). Ekkor a „**Custom webhook**” csatornát használd Adaptive Card törzsszel (lásd 6.5 hibaelhárítás).

Ha a feladat csak annyit kér, hogy „a riasztás menjen ki webhookon”, és kapsz egy URL-t, akkor a 6.2–6.4 elég. Ha magadnak kell webhookot generálni Teamsben, az alábbi 6.1.1 a recept.

6.1.1 Webhook létrehozása Teamsben (Workflows módszer)

1. Teamsben a cél **csatorna** mellett kattints a ... (**More options**) → **Workflows** (vagy bal sávban **Apps** → keresd a **Workflows** appot).
2. Válaszd a sablont: „**Post to a channel when a webhook request is received**”.
3. Lépj be a fiókkal (authentikáció), válaszd ki a **Team-et** és **Channel-t**.
4. **Add workflow** → kapsz egy **webhook URL-t** (ilyesmi: `https://prod-XX.westeurope.logic.azure.com:443/workflows/...`). **Másold ki.**
5. (Ha később újra kell az URL: Workflows app → a flow → **Edit** → nyisd ki a „When a Teams webhook request is received” triggert.)

 Tesztelheted SSH-ből curllel, hogy a webhook él-e (Workflows/Adaptive Card törzs):

```
bash

curl -H "Content-Type: application/json" -d '{
  "type": "message",
  "attachments": [{ "contentType": "application/vnd.microsoft.card.adaptive",
    "content": { "$schema": "http://adaptivecards.io/schemas/adaptive-card.json",
      "type": "AdaptiveCard", "version": "1.4",
      "body": [ { "type": "TextBlock", "text": "OpenSearch teszt üzenet" } ] } } ] }' "A_WEBHOOK_URL_IDE"
```

Ha megjelenik az üzenet a Teams csatornában, a webhook jó.

6.2 Notification Channel létrehozása az OpenSearch-ben

1. OpenSearch Dashboards → ≡ menü → **Notifications** → **Channels** → **Create channel**.
2. **Name:** pl. `teams-csatorna`.
3. **Channel type:**
 - (A) **klasszikus/megadott webhook esetén:** válaszd a „**Microsoft Teams**” típust → illeszd be a **webhook URL-t**.
 - (B) **Workflows webhook esetén, ha a natív nem megy:** válaszd a „**Custom webhook**” típust → **Method:** POST, **URL:** a webhook, **Header:** `Content-Type: application/json` (az üzenet törzsét a monitor action-jében adod meg, lásd 6.5).

4. (Lent „Availability”-nél hagyd az alapot, hogy az Alerting használhassa.)
5. **Send test message** → ellenőrizd, hogy megérkezik-e Teamsbe.
6. **Create**.

✅ **Ellenőrzés:** A teszt üzenet megjelenik a Teams csatornában. Ha nem → 6.5 hibaelhárítás.

6.3 Monitor (figyelő) létrehozása

1. ☰ menü → **Alerting** → **Monitors** → **Create monitor**.
2. **Monitor name:** pl. `hibas-keresek-monitor`.
3. **Monitor type:** a leggyakoribb a **Per query monitor** (lekérdezésenként).
4. **Monitor defining method:** **Visual editor** (a legkönnyebb).
5. **Frequency / Run every:** pl. **By interval** → **1 perc**.
6. **Data source:**
 - **Index:** `zh-logs-*`
 - **Time field:** `@timestamp`
7. **Query / Metric:**
 - Egyszerű darabszám-figyeléshez: a metrika **Count of documents**.
 - Szűréshez add hozzá a feltételt (pl. csak a `response >= 400` esetekre) – a vizuális szerkesztőben a **Query/filter** mezőben (pl. `response >= 400`), vagy aggregációval.
8. **Time range for the last:** pl. **5 perc** (mekkora ablakra nézze a feltételt).

6.4 Trigger + Action (riasztási feltétel + üzenetküldés)

1. A monitor szerkesztőjében **Add trigger**.
2. **Trigger name:** pl. `sok-hiba`.
3. **Severity level:** pl. **1 (Highest)**.
4. **Trigger condition:** pl. **IS ABOVE** és érték **10** (azaz ha 5 perc alatt 10-nél több hibás kérés van).
5. **Add action:**
 - **Action name:** pl. `teams-ertesites`.
 - **Channel:** válaszd a 6.2-ben létrehozott `teams-csatorna`-t.
 - **Message subject:** pl. `OpenSearch riasztás: sok hiba`.
 - **Message** (Mustache sablon – beilleszthetők változók):

```
A(z) {{ctx.monitor.name}} monitor riasztott.  
Trigger: {{ctx.trigger.name}} (súlyosság: {{ctx.trigger.severity}}).  
Időszak: {{ctx.periodStart}} - {{ctx.periodEnd}}.
```

- (Hasznos még: `{{ctx.results.0.hits.total.value}}` = a találatok száma.)

6. **Send test message** az action mellett → nézd meg, megjön-e Teamsbe.

7. **Create / Update**.

✓ Ellenőrzés:

- A monitor megjelenik az **Alerting → Monitors** listában, státusza fut.
- A teszt üzenet (vagy ha a feltétel teljesül, az éles riasztás) megérkezik a Teams csatornába.
- Ha akarsz „élesben” kiváltani: generálj annyi hibás eseményt a logból, ami átlépi a küszöböt (vagy állítsd a küszöböt alacsonyra, pl. IS ABOVE 0, hogy biztosan riasszon, majd nézd meg a Teams üzenetet).

6.5 🛠 Ha a Teams üzenet nem érkezik meg

- **A teszt curl Teamsbe megy, de az OpenSearch natív „Microsoft Teams” nem:** ez a Workflows-formátum különbség. Váltás „**Custom webhook**” csatornára, és az action **Message** törzsébe tedd az Adaptive Card JSON-t (a fenti 6.1.1 curl törzsével azonos szerkezet), a **Header** legyen `Content-Type: application/json`.
- **Semmi nem megy, a teszt curl sem:** a webhook URL rossz, lejárt, vagy a VM-nek nincs kifelé internetelérése a Teams/Azure felé. Ellenőrizd a hálózatot, és kérj/generálj új URL-t.
- **A csatorna nem jelenik meg a monitor „Channel” listájában:** mentsd el a csatornát újra (Notifications → Channels), frissítsd az oldalt; régebbi verziókban előfordult, hogy a Teams típusú csatorna nem renderelődött a trigger-felületen – ilyenkor a „Custom webhook” típus a biztos megoldás.

7. FELADAT: Forgatomelemzés (PCAP) – Wireshark

Ez független az OpenSearch-től. Nyisd meg a megadott `.pcap`/`.pcapng` fájlt **Wireshark**-ban (vagy használd `tshark`-ot parancssorból).

```
bash

wireshark /eleresi/ut/capture.pcap &
# vagy parancssorból gyors statisztika:
tshark -r capture.pcap -q -z io,phs          # protokoll-hierarchia
```

7.1 Általános „térkép” a forgalomról (mindig ezzel kezd)

A **Statistics** menüből:

- **Protocol Hierarchy** → milyen protokollok, milyen arányban (HTTP, DNS, TLS, FTP, stb.).
- **Conversations** → ki kivel beszélt (IP-párok), melyik a legtöbb csomag/byte („top talkers”). A **TCP/UDP** fülön a portokat is látod.
- **Endpoints** → mely IP-k szerepelnek, melyik a legaktívabb.

- **Capture File Properties** (Statistics menü) → összes csomag száma, időtartam, capture kezdete/vége.

7.2 Display filterek (a keresősáv felül) - a leghasznosabbak

Cél	Display filter
Csak HTTP	<code>http</code>
Csak DNS	<code>dns</code>
Adott IP (forrás vagy cél)	<code>ip.addr == 10.0.0.5</code>
Forrás IP	<code>ip.src == 10.0.0.5</code>
Cél IP	<code>ip.dst == 10.0.0.5</code>
Adott port	<code>tcp.port == 80</code>
HTTP kérés metódusa	<code>http.request.method == "POST"</code>
HTTP státuszkód	<code>http.response.code == 200</code>
URL/hoszt szűrés	<code>http.host contains "example"</code>
DNS lekérdezett név	<code>dns.qry.name contains "phish"</code>
Szöveg a teljes csomagban	<code>frame contains "password"</code>
FTP parancsok	<code>ftp</code>
TCP SYN (kapcsolat-kezdemények / port scan jele)	<code>tcp.flags.syn == 1 && tcp.flags.ack == 0</code>
TLS „Client Hello” (SNI - mely domainekhez ment HTTPS)	<code>tls.handshake.type == 1</code>

Szűrő alkalmazása után **jobb klikk egy csomagon** → **Apply as Column** segít az értékek táblázatos megjelenítésében.

7.3 Mélyebb vizsgálat

- **Tartalom (jelszó, üzenet) kiolvasása:** jobb klikk egy csomagon → **Follow** → **TCP Stream** (vagy HTTP/UDP Stream). Itt összeáll a teljes beszélgetés (pl. egy HTTP POST body, egy FTP `(USER)`/`(PASS)`). Titkosítatlan protokolloknál (HTTP, FTP, Telnet) így látszanak a hitelesítő adatok.
- **Átküldött fájlok kinyerése:** **File** → **Export Objects** → **HTTP** (vagy SMB/FTP-DATA) → látod és mentheted a letöltött fájlokat (képek, exe, dokumentumok).

- **HTTPS célok (domainek) titkosítás nélkül is:** a `tls.handshake.type == 1` szűrővel a **Server Name (SNI)** mezőből kiolvasható, melyik domainekhez ment HTTPS forgalom.
- **TCP visszafejtés/anomáliák:** `tcp.analysis.flags` (újraküldések, hibák).

7.4 Tipikus ZH-kérdések és hol a válasz

Kérdés	Hol nézd
Hány csomag van összesen?	Statistics → Capture File Properties (vagy a státuszsor jobb alja).
Melyik két gép kommunikált a legtöbbet?	Statistics → Conversations, rendezés Bytes/Packets szerint.
Milyen protokollok vannak?	Statistics → Protocol Hierarchy.
Milyen DNS neveket kérdeztek le?	Szűrő: <code>dns.qry.name</code> , vagy Statistics → DNS.
Történt-e bejelentkezés / mi a jelszó?	<code>http.request.method == "POST"</code> vagy <code>ftp</code> / <code>telnet</code> → Follow Stream.
Milyen fájl(oka)t töltöttek le?	File → Export Objects → HTTP.
Melyik User-Agent / böngésző?	<code>http.user_agent</code> (Apply as Column).
Volt-e portszkennelés?	<code>tcp.flags.syn==1 && tcp.flags.ack==0</code> → sok cél-port egy IP-ről egy célra.
Mely HTTPS oldalakra ment a forgalom?	<code>tls.handshake.type == 1</code> → SNI mező.
Mikor kezdődött / meddig tartott?	Statistics → Capture File Properties (idők).

7.5 tshark gyors parancsok (ha a CLI gyorsabb)

```
bash
```

```
# Összes csomag száma
tshark -r capture.pcap | wc -l

# Csak HTTP kérések URL-jei
tshark -r capture.pcap -Y "http.request" -T fields -e http.host -e http.request.uri

# DNS lekérdezett nevek
tshark -r capture.pcap -Y "dns.flags.response==0" -T fields -e dns.qry.name | sort |

# Top forrás IP-k
tshark -r capture.pcap -T fields -e ip.src | sort | uniq -c | sort -rn | head

# TLS SNI (HTTPS céldomainek)
tshark -r capture.pcap -Y "tls.handshake.type==1" -T fields -e tls.handshake.extensions
```

👉 Írd le minden kérdés mellé a választ (és hogy hol/melyik szűrővel kaptad), hogy ellenőrizhető legyen.

8. Hibaelhárítás - gyors gyűjtemény szakaszonként

Tünet	Valószínű ok	Megoldás
Filebeat: <code>could not parse config</code>	YAML behúzási hiba / TAB	Csak szóköz, ellenőrizd a behúzást; <code>filebeat test config</code> .
<code>filebeat test output</code> nem OK	Logstash nem fut / 5044 zárt	<code>systemctl status logstash</code> , <code>ss -tlnp grep 5044</code> .
Filebeat fut, de nincs adat	A registry miatt nem küld újra	2.3: registry törlés + újraindítás.
Logstash: <code>Unable to connect to OpenSearch</code> / 401	rossz jelszó / port / HTTPS	<code>user</code> / <code>password</code> javítása, <code>ssl => true</code> , <code>ssl_certificate_verification => false</code> , port 9200.
Logstash: <code>output-opensearch</code> plugin nincs	plugin hiányzik	<code>logstash-plugin install logstash-output-opensearch</code> .
<code>tags: grokparsefailure</code>	a grok minta nem illeszkedik	A <code>rubydebug</code> / online debugger alapján faragd a mintát; ellenőrizd a fix elhatárolókat (szóköz, <code>[]</code> , idézőjel).
OpenSearch index létrejön, de a számok stringek	grok minden mezője string	<code>mutate { convert => { "mező" => "integer" } }</code> .

Tünet	Valószínű ok	Megoldás
Dashboards: „No results found”, pedig van adat	rossz időablak / <code>@timestamp</code> a beolvasás ideje	Bővítsd az időablakot (5.2); állítsd be a <code>date</code> filtert (3.5).
Index pattern nem találja az indexet	nincs még adat / rossz minta	Előbb küldj be adatot (2.); a mintában legyen csillag: <code>zh-logs-*</code> .
Teams: nem jön az üzenet, a teszt curl sem	rossz/lejárt URL vagy nincs kifelé net	Új Workflows URL; hálózat ellenőrzése.
Teams: teszt curl megy, OpenSearch nem	natív „Microsoft Teams” vs Workflows formátum	„Custom webhook” + Adaptive Card törzs (6.5).
Bármi „connection refused” 9200-on	OpenSearch még indul / nem fut	Várj 1-2 percet; <code>systemctl status opensearch</code> ; nézd: <code>/var/log/opensearch/</code> .

Hol vannak a logok (hibakereséshez):

```
bash

sudo tail -f /var/log/logstash/logstash-plain.log
sudo journalctl -u filebeat -f
sudo tail -f /var/log/opensearch/*.log
sudo tail -f /var/log/opensearch-dashboards/*.log # ha létezik
```

9. Gyors parancs-referencia (puska)

```
bash
```

```
# --- Szolgáltatások ---
sudo systemctl restart logstash
sudo systemctl restart opensearch
sudo systemctl restart opensearch-dashboards

# --- Filebeat: teszt + futtatás előtérben ---
sudo filebeat test config
sudo filebeat test output
sudo filebeat -e -c /etc/filebeat/filebeat.yml

# --- Filebeat ÚJRAKÜLDÉS (registry reset) ---
sudo systemctl stop filebeat
sudo rm -rf /var/lib/filebeat/registry
sudo filebeat -e -c /etc/filebeat/filebeat.yml

# --- Logstash: config teszt + futtatás előtérben ---
sudo -u logstash /usr/share/logstash/bin/logstash --path.settings /etc/logstash \
  -f /etc/logstash/conf.d/pipeline.conf --config.test_and_exit
sudo -u logstash /usr/share/logstash/bin/logstash --path.settings /etc/logstash \
  -f /etc/logstash/conf.d/pipeline.conf

# --- Logstash plugin ellenőrzés/telepítés ---
sudo /usr/share/logstash/bin/logstash-plugin list | grep opensearch
sudo /usr/share/logstash/bin/logstash-plugin install logstash-output-opensearch

# --- OpenSearch ellenőrzések ---
curl -k -u admin:admin "https://localhost:9200/_cat/indices?v"
curl -k -u admin:admin "https://localhost:9200/zh-logs-*/_search?pretty&size=1"
curl -k -u admin:admin "https://localhost:9200/zh-logs-*/_count?q=tags:_grokparsefail"
curl -k -u admin:admin -X DELETE "https://localhost:9200/zh-logs-*" # index törlés

# --- PCAP gyors statisztikák ---
tshark -r capture.pcap -q -z io,phs
tshark -r capture.pcap -T fields -e ip.src | sort | uniq -c | sort -rn | head
```

10. Vizsga-előtti checklist (1 perc alatt fúsd át)

- ☐ Tudom az OpenSearch **jelszót** és be tudok lépni a Dashboardsra (`:5601`).
- ☐ Tudom a **log-fájlok elérési útját** (feladatból).
- ☐ Tudom a **pipeline.conf** és a **filebeat.yml** helyét.
- ☐ Emlékszem a **lánc ellenőrzésére**: `_cat/indices` → van index + doc.
- ☐ Emlékszem a **registry reset** 3 lépésére (újraküldéshez).
- ☐ Tudom, hogy „nincs adat a Discoverben” → **időablak** + `date` filter.

- ☐ Tudom, hogy a státuszkód-szűréshez kell `mutate convert integer`.
- ☐ Index pattern: `zh-logs-*`, time field `@timestamp`.
- ☐ Teams: ha a natív nem megy → **Custom webhook + Adaptive Card**.
- ☐ PCAP-nél előbb: **Protocol Hierarchy + Conversations**, aztán célzott szűrők.

Sok sikert! Haladj fentről lefelé, és minden szakasz után csak akkor lépj tovább, ha a  **Ellenőrzés** sikerült.